ACTIVITAT AVALUABLE AC1

Mòdul: Desplegament d'aplicacions web

Professor: Marc Callejón

Data límit d'entrega: 05/04/2026 23:59

Mètode d'entrega: Per mitjà del Clickedu de l'assignatura. Les activitats entregades més enllà de la data límit només podran obtenir una nota de 5.

Es valorará la presentació.

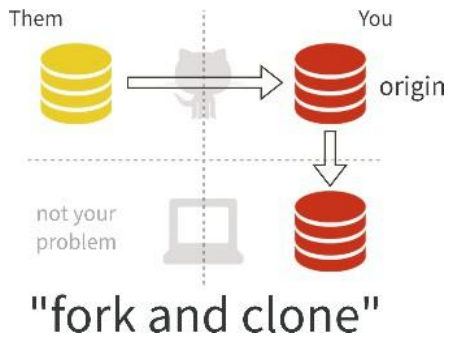
En esta práctica exploraremos algunos aspectos importantes del <u>despliegue *online* de un proyecto</u>. Para ello, utilizaremos el proyecto en NodeJS que encontraréis en el repositorio de más abajo

Es importante, para el correcto desempeño de la práctica, documentar todos los pasos del proceso.

<u>Despliegue del proyecto en local</u>

Tarea 1. Realiza un *fork* del siguiente repositorio a tu GitHub, y clona este a tu máquina local.

https://github.com/marcetpx/M-AC1API/tree/main



El proyecto que encontrarás en el repositorio necesita de dos pasos previos antes de poder ser ejecutado en local.

a. **Dado que utiliza una conexión a MongoDB, tendrás que configurar una cuenta, un clúster y una base de datos en el *cloud* Atlas de MongoDB (tarea 2).**
b. **Dado que no se incluyen las dependencias (módulos de *Node*) en el repositorio, se deberán instalar manualmente mediante *npm* (tarea 3).**

Tarea 2. Sigue las instrucciones que encontrarás en el enlace siguiente para crear la base de datos en Atlas.

What is MongoDB Atlas?

MongoDB Atlas is a multi-cloud database service by the same people that build MongoDB. Atlas simplifies deploying and managing your databases while offering the versatility you need to build resilient and performant global applications on the cloud providers of your choice.

https://www.mongodb.com/docs/atlas/getting-started/

Ten en cuenta que, para que nuestra aplicación pueda utilizar el clúster debes asegurarte de hacer las siguientes operaciones:

a. **Crear un base de datos llamadas *products*. Las colecciones que contenga (el análogo a las tablas en el modelo relacional), las creará la propia aplicación.**
b. **Crear un nuevo usuario. Es recomendable que la contraseña asociada la genere automáticamente la UI de Atlas. Asegúrate de que el nuevo usuario creado dispone de todos los permisos sobre la base de datos.**
c. **Añadir la IP desde la que vas a trabajar a la *whitelist* del clúster (en la UI de Atlas: *Data Services/Security/Network Access*). Puedes saber la IP pública desde la que te estas conectando con esta herramienta, por ejemplo: https://www.whatismyip.com/es/.**

-Finalmente, añade la URI de la connexión (la UI de Atlas: *Data Services/Database/Connect*) al fichero */config/config.js* del proyecto.

Tarea 3. Como puedes observar, el repositorio no contiene la carpeta de dependencias *node_modules*. Esta es una buena práctica en tiempo de desarrollo: ¿para qué subir los paquetes de dependencias, que son de largo los más pesados del proyecto, al repositorio (o a Clickedu...), si puede reconstruirse a partir del fichero *package.json*?

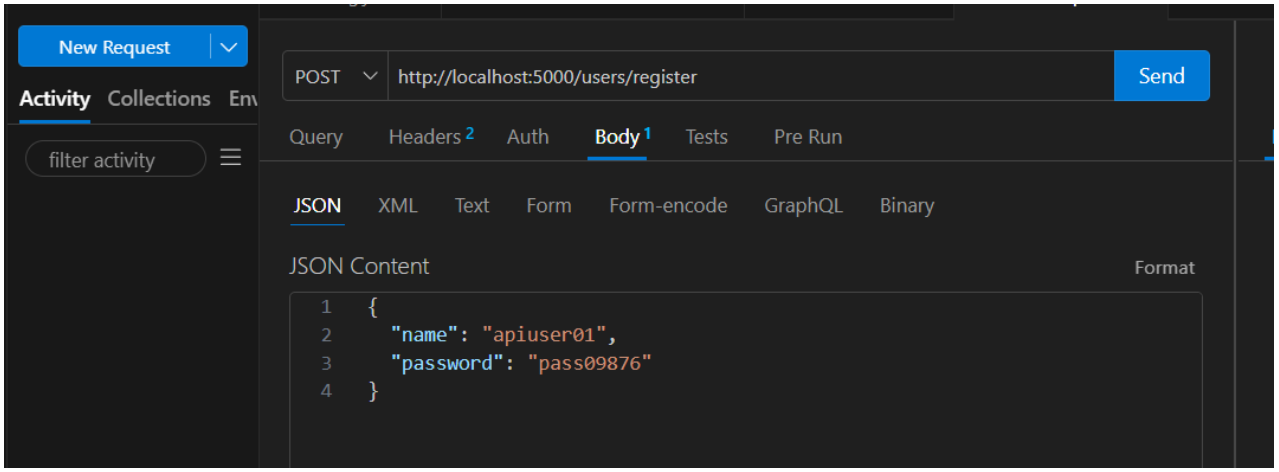
- **a) Observa cómo hemos evitado que las dependencias se añadan al control de versiones añadiendo la carpeta *node_modules* al fichero *.gitignore*.**
- **b) Observa cómo las dependencias del proyecto están listadas en el fichero *package.json*. Es importante tener este archivo *mantenido*. Si dejamos de utilizar un paquete, hay que *desinstalarlo* para que desaparezca tanto de nuestro directorio de dependencias como del fichero *package.json*.**
- **c) Instala las dependencias con *npm install*.**
- **d) Ejectuta la aplicación (p.ej. *nodemon* app.js) y comprueba que la página de bienvenida de la API se sirve en *http://localhost:5000/api***

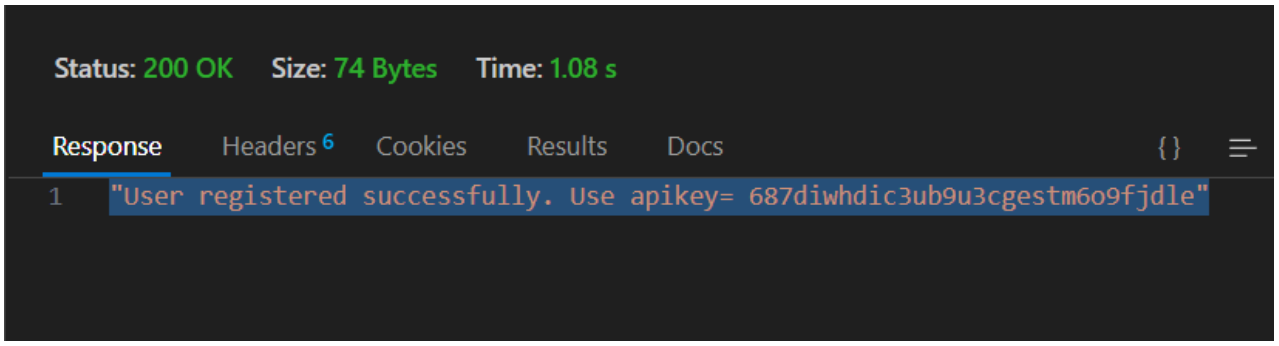
Verás que en la página no aplican estilos. Nos ocuparemos de esto más adelante.

- **e) Documenta el correcto funcionamiento de la aplicación:**

-Registra un nuevo usuario (observa el formato del JSON):

Para ello he usado la extension del vsc code Thunder Client





POST http://localhost:5000/api/products Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "id": "1",
3   "name": "Ramo44000",
4   "price": 600
5 }
```

Status: 200 OK Size: 30 Bytes Time: 1.09 s

Response Headers 6 Cookies Results Docs

```
1 "Product created successfully"
```

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "id": "2",
3   "name": "Laptop5000",
4   "price": 1200
5 }
```

Status: 200 OK Size: 30 Bytes Time: 103 ms

Response Headers 6 Cookies Results Docs

```
1 "Product created successfully"
```

JSON Content

```
1 {
2   "id": "3",
3   "name": "Monitor4K",
4   "price": 350
5 }
6
```

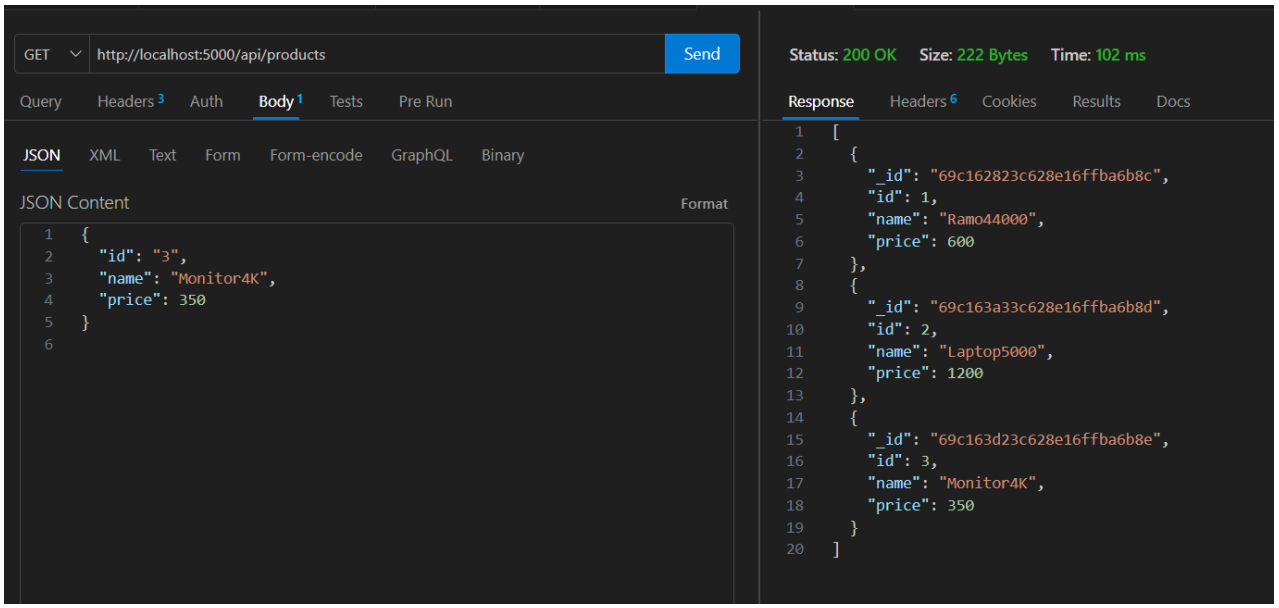
Status: 200 OK Size: 30 Bytes Time: 97 ms

Response Headers 6 Cookies Results Docs

```
1 "Product created successfully"
```

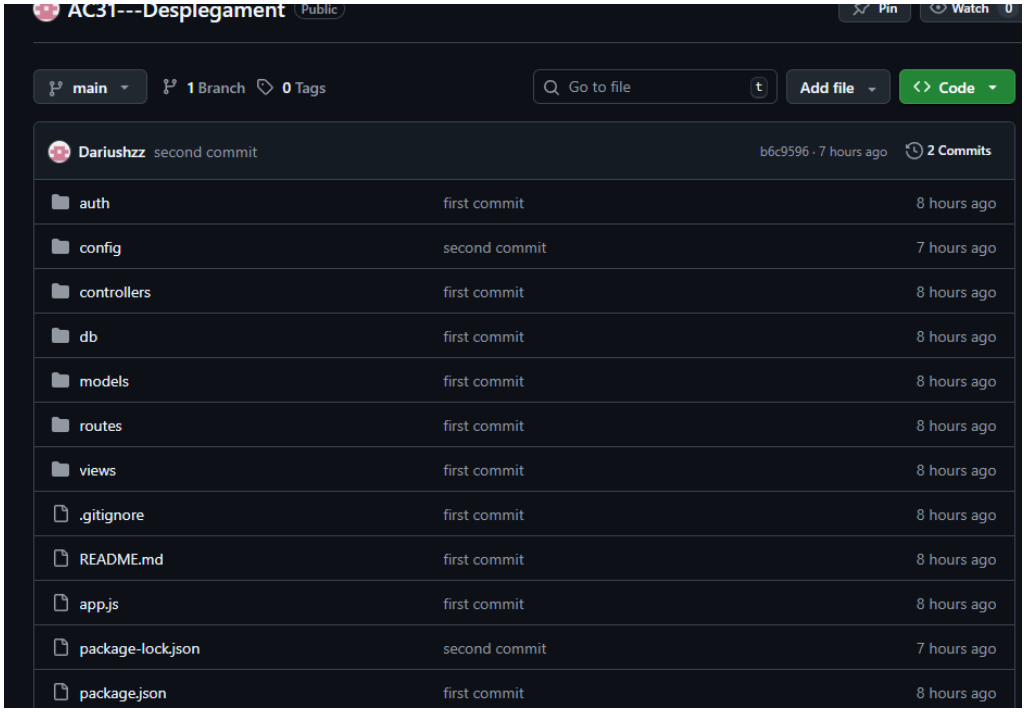
-Verás que en la respuesta HTTP se proporciona una apikey:

-Crea algunos productos de prueba:



-Consulta la lista de productos

f) Finalmente, sube los cambios del proyecto a tu repositorio. Sólo debería haber cambiado el *config.js* (del código fuente). Asegúrate de que no en el *.gitignore* siguen estando la carpeta de dependencias y el fichero CSS.



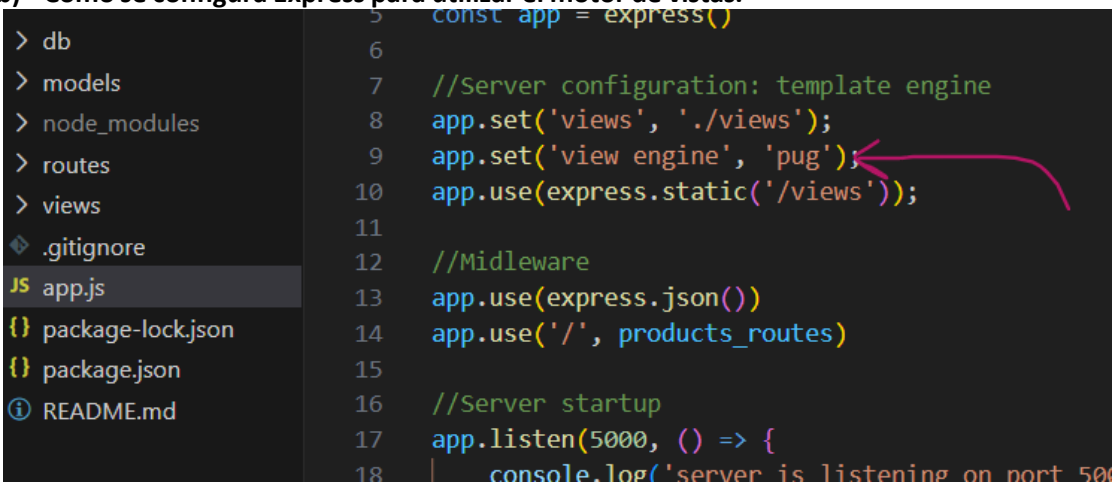
Tarea 3. Observa que el proyecto usa una VISTA renderizada por el backend, disponible en el endpoint */api/*.

Investiga en el código del backend:

a) Qué motor de plantillas se utiliza.

PUG. Podemos ver index.pug dentro de styles que a su vez esta dentro de views

b) Cómo se configura Express para utilizar el motor de vistas.



-Despliegue del proyecto online (Render)

Tarea 4. A continuación, desplegaremos el proyecto *online*. Una buena solución para un aplicación NodeJS sería utilizar el *hosting* de Heroku, pero recientemente han retirado su *free tier*. Por ello, utilizaremos una alternativa parecida:

https://render.com/

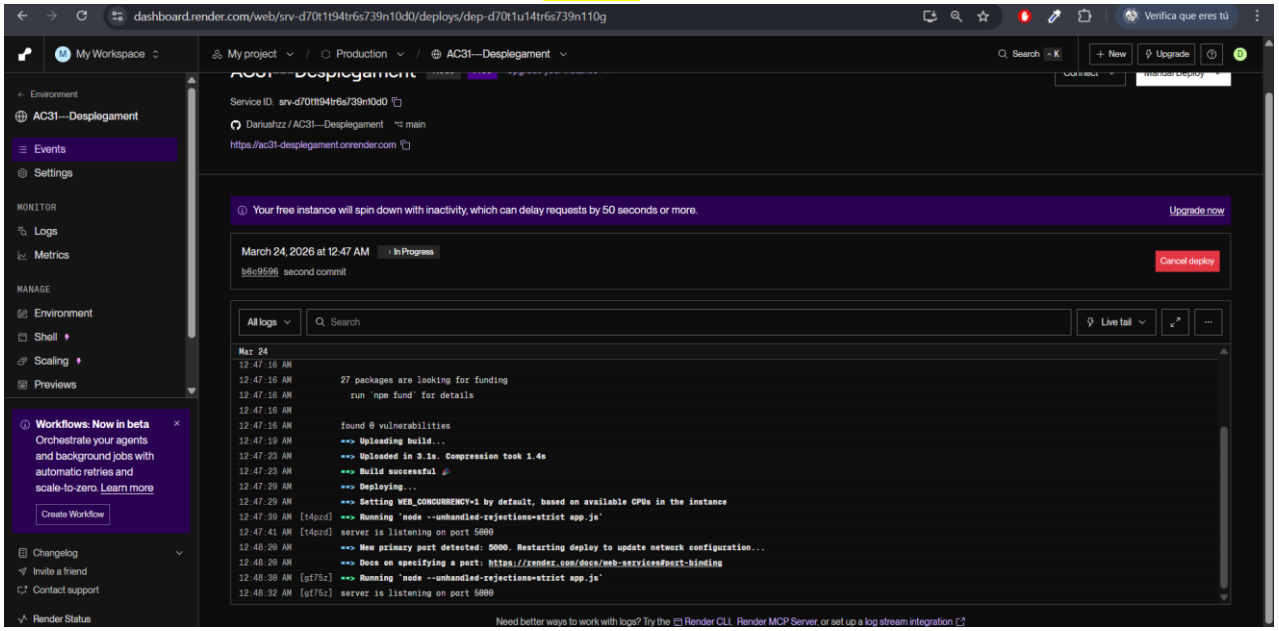
Sigue las instrucciones en este *post* para crear una cuenta en *render.com*, conectarla con tu cuenta de *GitHub* y desplegar el último *commit* realizado.

https://www.freecodecamp.org/news/how-to-deploy-nodejs-application-with-render/

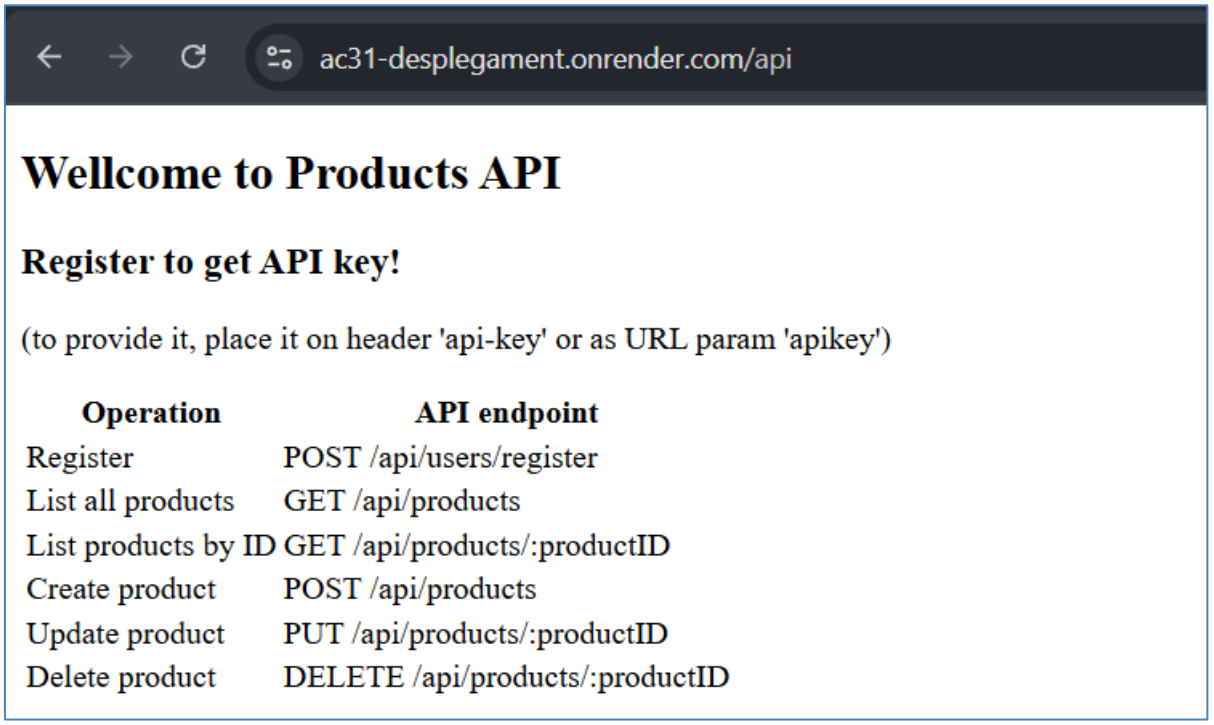
Durante el proceso de instalación, asegúrate de:

a) **Configurar como comando de compilación (*build command*): *npm install*, ya que las dependencias no se incluyen en el repositorio y deben instalarse en tu sesión del *hosting*.**
b) **Configurar como comando de arranque (*start command*):**
   node --unhandled-rejections=strict app.js

La opción *unhandled-rejection=strict* soluciona algunos problemas derivados del uso que hacemos del paquete mongodb en relación con la versión de NodeJS que hay instalada en el *hosting*.

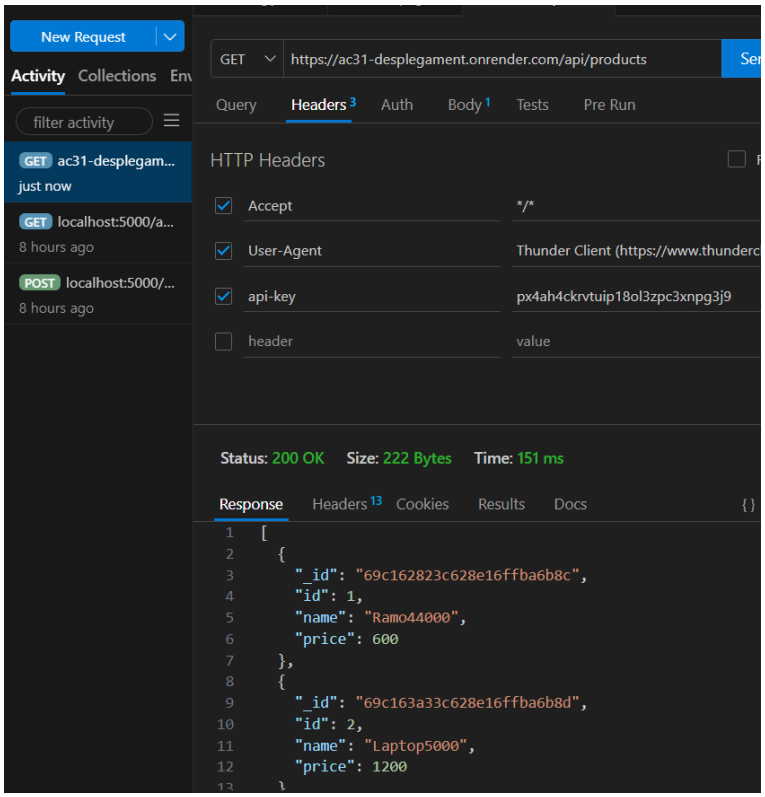


c) **Observa como, después de configurar el nuevo servicio web, se desplega (*deploy*) automáticamente el proyecto del repositorio connectado, en la versión del último commit (si has marcado la opción *autodeploy* durante el proceso de configuración). Captura el *log* del proceso de *deployment*.**
d) **Comprueba el funcionamiento de la api: lo normal sería que pudieras acceder a la página de bienvenida (sin estilos), pero que no pudieras utilizar correctamente el resto de *endpoints*.**



e) **Para que éstos funcionen, deberás añadir las IPs del servicio web a la *whitelist* de Atlas. Puedes saber cuáles son las IPs públicas del servicio web en el botón *Connect* del panel de control principal del servicio.**

f) **Comprueba el correcto funcionamiento de la API, de manera exhaustiva.**

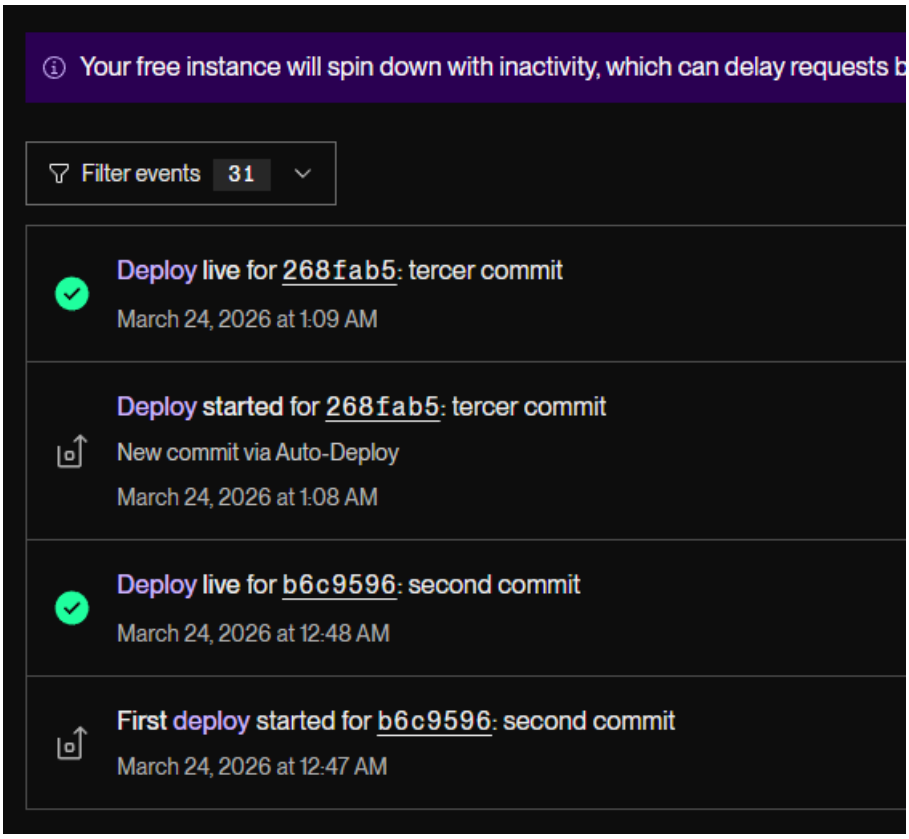


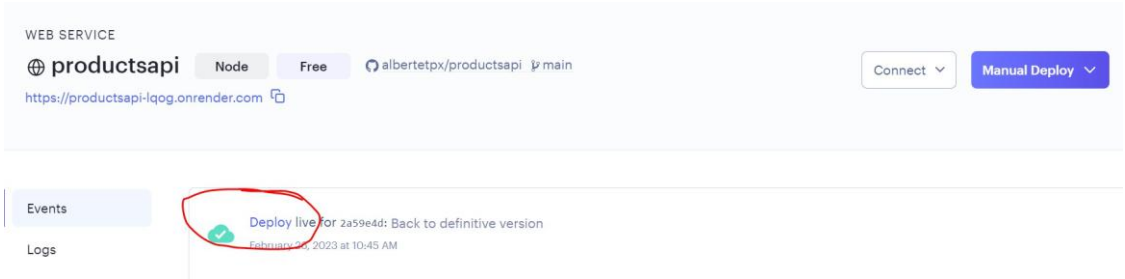
Tarea 5. El hecho de que el web service se compile y despliegue automáticamente desde el repositorio de *GitHub* facilita enormemente el proceso de actualización del proyecto. Hagamos un cambio tonto para tener claro el proceso:

- ***a)*** **Instala en el proyecto de tu máquina local la dependencia *slugify*. Observa como se ha añadido al *package.json*.**
- **b) Usa *slugify* para *slugizar* con asteriscos el mensaje de bienvenida del servidor Express (en *app.js*).**

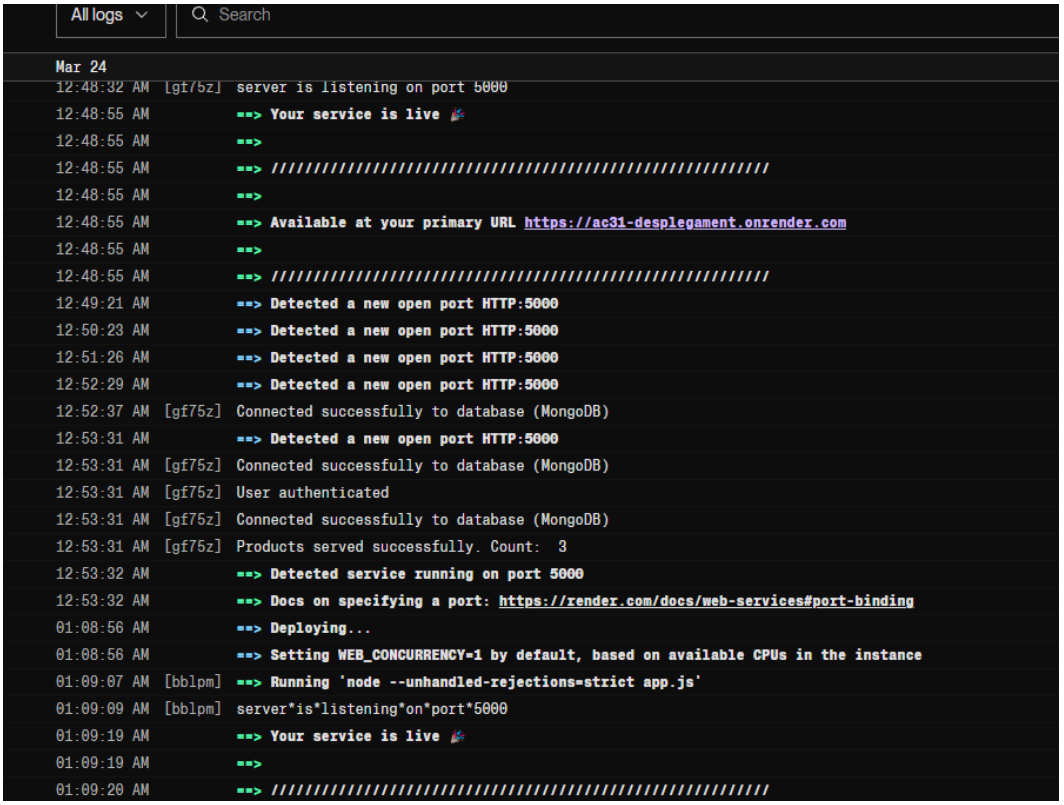
```
C:\Users\Dariush\Documents\2n DAW\Marc - Despliegue aplicaciones web\UF3 ---
-------------------\AC31---Desplegament>node app.js
(node:3172) [DEP0040] DeprecationWarning: The `punycode` module is deprecate
d. Please use a userland alternative instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
server*is*listening*on*port*5000
```

- **c) Sube cambios a tu repositorio e *GitHub*.**
- **d) En *render,* comprueba cómo se lanza automáticamente el despligue del *commit* que acabas de realizar. Pueder acceder a las trazas del proceso de compilacion y despliegue clicando sobre el *deploy* que toque en el *dashboard* de tu servicio web:**





e) **Accede a la consola del servicio web (sección *logs* del *dashboard*) para comprobar que se han aplicado los cambios correctamente (*slugización* del *prompy* de bienvenida del servidor Express).**



-Automatización de la compilación

Tarea 6. Habitualmente, el proceso de despliegue de una aplicación web habrá de venir precedido no solo de la instalación de las dependencias, sino de la compilación de ciertos ficheros. Estos son algunos ejemplos:

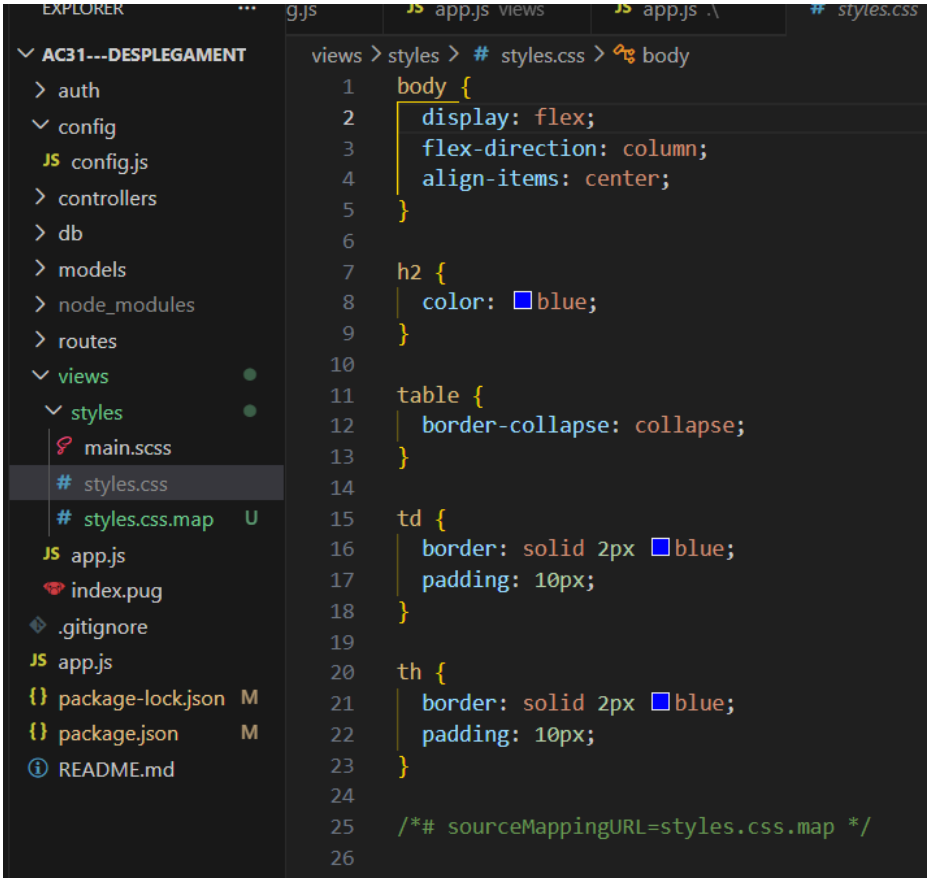
-Compilación de SCSS a CSS.
-Compilación de *frameworks* JS: REACT, Vue...
-Transpilación de Typescript a Javascript (en Angular, por ejemplo).
-Post-procesado de CSS (autoprefixing, por ejemplo).
-Minificación de ficheros.
-Linting de código.

En esta ocasión, necesitamos automatizar el proceso de compilación de CSS a partir de ficheros SASS. Para ello:

a) **Instala el paquete *node-sass* en tu proyecto local, como <u>depedencia de desarrollo</u>: *npm install --save-dev node-sass*. Observa como se añade al *package.json.***

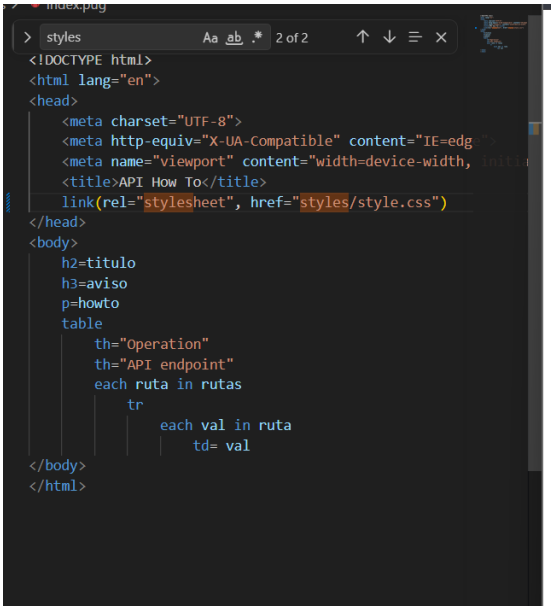
b) **Lanza la compilación de estilos mediante *npx:***

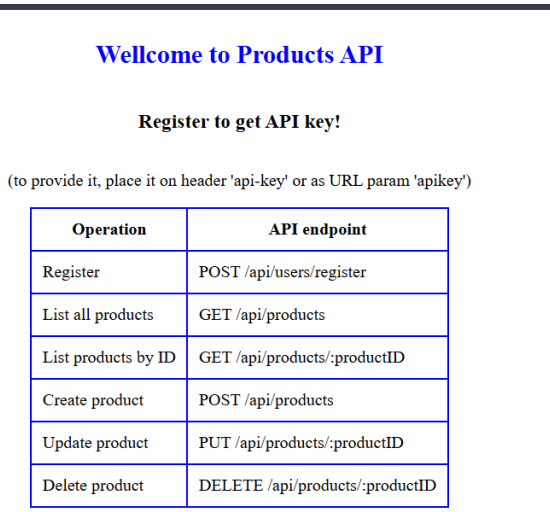
npx node-sass ./views/styles/main.scss ./views/styles/styles.css



c) **Observa cómo se genera el fichero de estilos. Ahora la página de bienvenida de la API aparecerá correctamente estilada.**

El index.pug tiene un fallo en la etiqueta style. Pero arreglandolo con la IA funciona





d) **Convertiremos ahora el comando de compilación de estilos en un *script npm*. Para ello, añade una nueva clave en el apartado de *scripts* del *package.json*:**

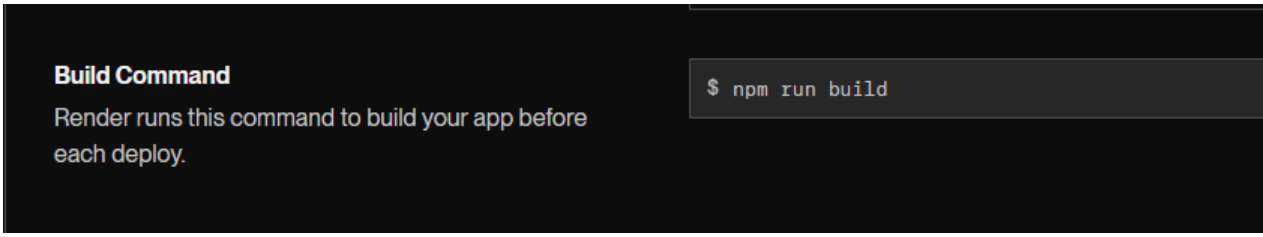
"build:css": "node-sass ./views/styles/main.scss ./views/styles/styles.css"

e) **Haz algún cambio en el SASS y recompila lo estilos, esta vez usando el script npm: npm run build:css.**

f) **Finalmente, empaquetaremos las dos operaciones (instalación de depencias y compilación de estilos) en un único *script npm*:**

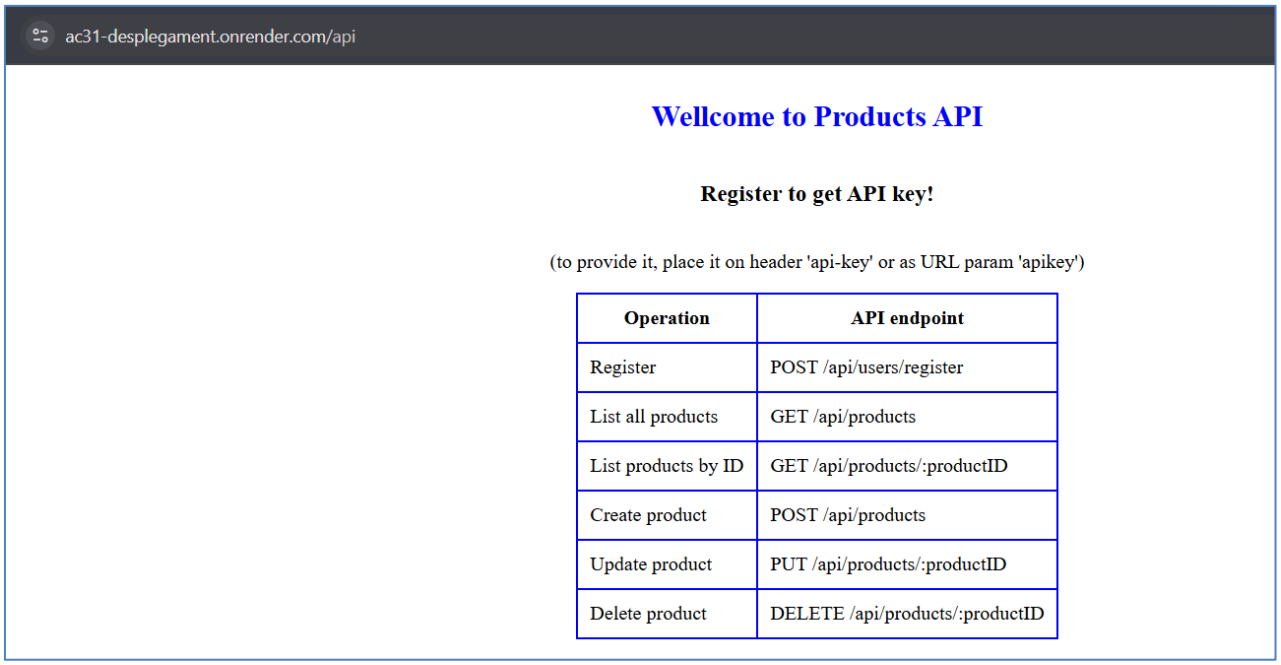
"build": "npm install && npm run build:css",

g) **Actualiza el compando de compilación (*build command*) en los *settings* del servicio web en render a: *npm run build*.**



h) **Sube los campios al repo, observa cómo se lanza un nuevo despliegue en render, y como, ahora sí, se genera el fichero de estilos (que, recordemos, NO SE SUBE AL REPO), y la página de bienvenida de la API *online* aparece estilada correctamente.**

En el siguiente enlace puedes ver otros *scripts npm* útiles para la automatización de procesos de despliegue:
https://css-tricks.com/why-npm-scripts/



Operation	API endpoint
Register	POST /api/users/register
List all products	GET /api/products
List products by ID	GET /api/products/:productID
Create product	POST /api/products
Update product	PUT /api/products/:productID
Delete product	DELETE /api/products/:productID

-Ocultación de secretos:

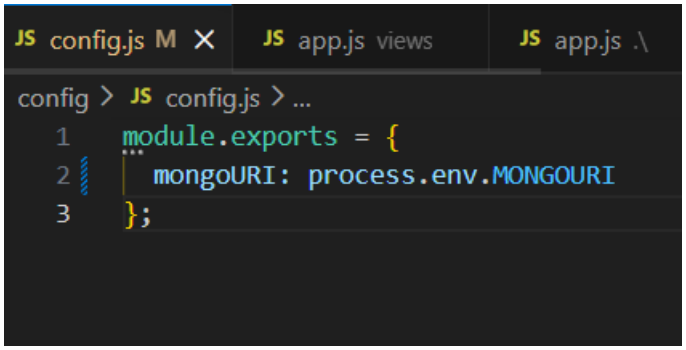
Tarea 7. Lo más probable es que *GitHub* te haya avisado, mediante un correo, de que existe en tu repositorio una vulnerabilidad de seguridad. Concretamente, de que la URI de la conexión a la base de datos de MongoDB está expuesta, en texto plano, en el repositorio. Esto es gravísimo.

a) **Vamos a convertir la URI a una variable de entorno para que no quede almacenada en el código. Para ello, desde la terminal integrada de VSCode, usa:**

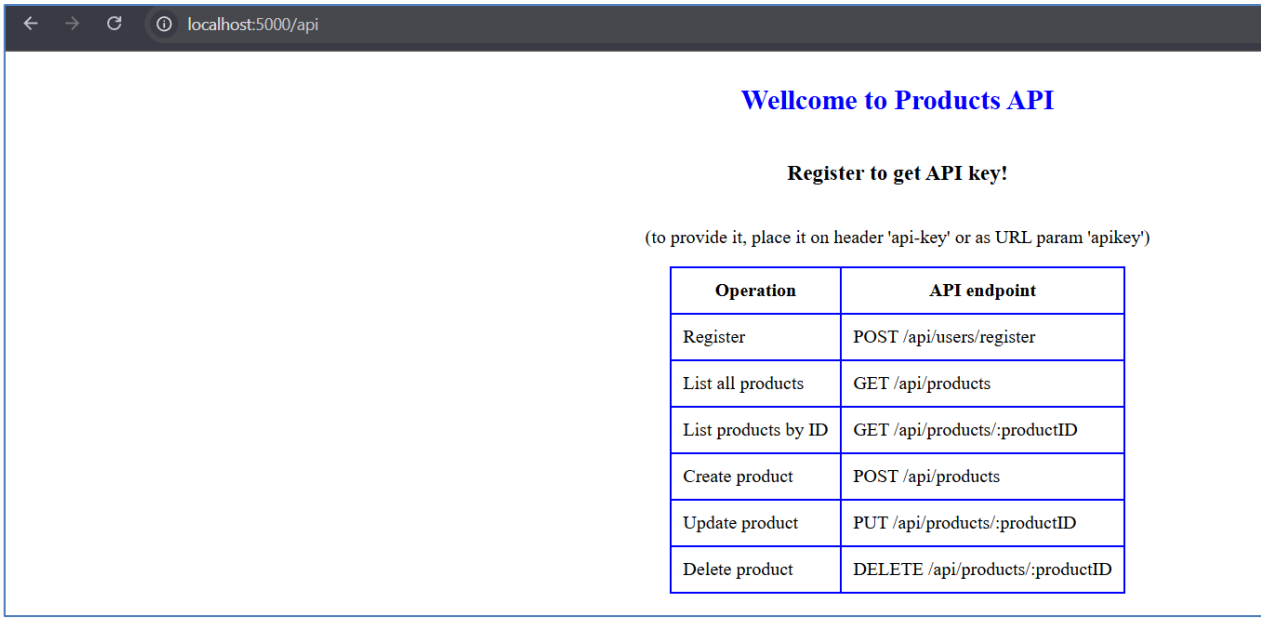
$ENV:MONGOURI='tuURI'

b) **Ahora, haz que en el fichero *config.js* se lea esta URI desde la variable de entorno:**

process.env. MONGOURI;



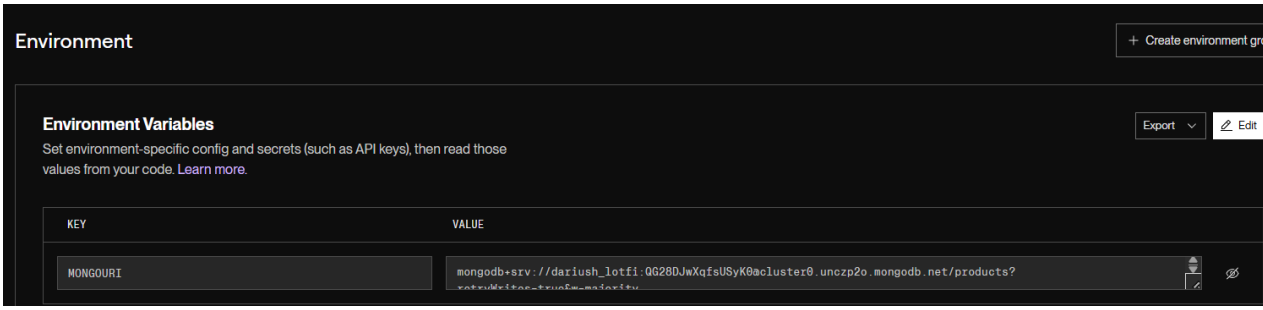
c) **Comprueba que la API sigue pudiendo conectar correctamente a la base de datos.**



Operation	API endpoint
Register	POST /api/users/register
List all products	GET /api/products
List products by ID	GET /api/products/:productID
Create product	POST /api/products
Update product	PUT /api/products/:productID
Delete product	DELETE /api/products/:productID

d) **En el panel de control del servicio web en render, en la sección *Environment*, añade la variable de entorno a la sesión del *hosting*.**

e) **Sube los cambios locales al repositorio, observa como se lanza otro *deploy*, y comprueba que la API *online* sigue funcionando. La URI de MongoDB ha dejado de estar expuesta en el repositorio.**



Esta manera de trabajar con variables de entorno no es la óptima. Existen algunos paquetes (p.ej. *dotenv*) que automatizan la configuración en el entorno de estas variables a partir de ficheros dedicados de extensión *.env*, que OBVIAMENTE, deben añadirse al *.gitignore*, para no quedar expuestos en el repo, y subirse a mano al *hosting*. Investiga en qué lugar se suben este tipo de ficheros en el caso de render.